BASES DE DATOS II

Universidad del Quindío

Ingeniería de sistemas y computación

Armenia, Quindío



Guia de ejecución

Daniel Esteban Cifuentes Blandón

1094963882

2026

TurismoUQ

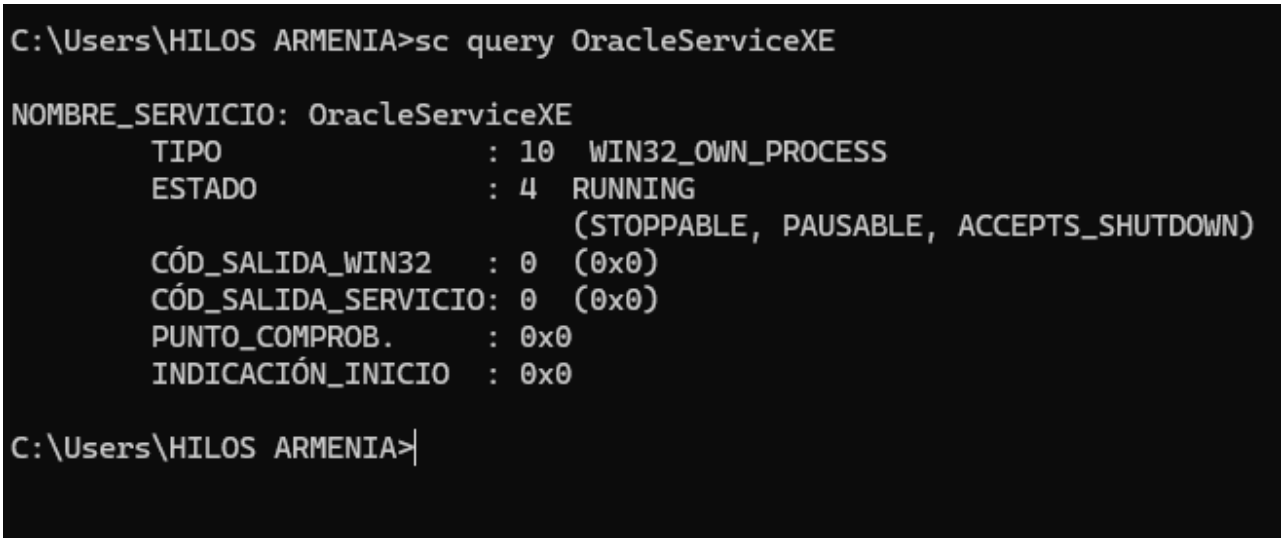
0. Datos de acceso

Dato	**Valor**
Motor	Oracle Database 21c Express Edition (instalación local en Windows)
Ruta de sqlplus	C:\OracleDatabase\dbhomeXE\bin\sqlplus.exe
Usuario SYS (administrador Oracle)	sys / Daniel.sharond15
Usuario del esquema del proyecto	turismouq / Daniel.sharond15
PDB (base conectable)	XEPDB1
Cadena de conexión	localhost:1521/XEPDB1
Carpeta de scripts	C:\Users\HILOS ARMENIA\Desktop\PROYECTO_BASES_DATOS\scripts
Tablespace del histórico	TS_HISTORICO_RESERVAS

1. Verificar que el motor está arriba.

Antes de conectarse hay que confirmar que el servicio de Oracle está corriendo. En CMD:

sc query OracleServiceXE



2. Conectarse como administrador.

"C:\OracleDatabase\dbhomeXE\bin\sqlplus.exe" sys/Daniel.sharond15@localhost:1521/XEPDB1 as sysdba

```
C:\Users\HILOS ARMENIA>"C:\OracleDatabase\dbhomeXE\bin\sqlplus.exe" sys/Daniel.sharond15@localhost:1521/XEPDB1 as sysdba

SQL*Plus: Release 21.0.0.0.0 - Production on Vie Sep 25 07:51:42 2026
Version 21.3.0.0.0

Copyright (c) 1982, 2021, Oracle.  All rights reserved.

Conectado a:
Oracle Database 21c Express Edition Release 21.0.0.0.0 - Production
Version 21.3.0.0.0
```

3. Confirmar en qué base estoy parado.

SELECT USER, SYS_CONTEXT('USERENV','CON_NAME') AS contenedor FROM dual;

```
SQL> SELECT USER, SYS_CONTEXT('USERENV','CON_NAME') AS contenedor FROM dual;

USER
--------------------------------------------------------------------------------
CONTENEDOR
--------------------------------------------------------------------------------
SYS
XEPDB1
```

4. Ejectuar el Script 00 (tablespace y usuario)

@"C:\Users\HILOSARMENIA\Desktop\PROYECTO_BASES_DATOS\scripts\00_tablespace.sql"

```
SQL> @"C:\Users\HILOS ARMENIA\Desktop\PROYECTO_BASES_DATOS\scripts\00_tablespace.sql"
Procedimiento PL/SQL terminado correctamente.

Tablespace creado.

Procedimiento PL/SQL terminado correctamente.

Usuario creado.

Concesi¾n terminada correctamente.

Concesi¾n terminada correctamente.

Concesi¾n terminada correctamente.

Concesi¾n terminada correctamente.
```

```
TABLESPACE_NAME
------------------------------
FILE_NAME                                                          MB
------------------------------------------------------------ --------
TS_HISTORICO_RESERVAS
C:\ORACLEDATABASE\ORADATA\XE\XEPDB1\TS_HISTORICO_RESERVAS01.DB       64
F

USERS
C:\ORACLEDATABASE\ORADATA\XE\XEPDB1\USERS01.DBF                      25
```

Creamos un espacio físico donde vivirán los datos y el usuario dueño.
Que hace el comando:
@ruta\00_tablespace.sql : le dice a SQL*Plus "lee ese archivo y ejecuta linea por linea lo que hay dentro".
El script hace tres cosas: crea el tablespace TS_HISTORICO_RESERVAS (archivo de 6;B que crece solo de a 32MB hasta un tope que de 2GB, crea el usuario turismouq con cuota en los dos tablespacs, y le otorga 4 permisos.
El resultado son dos tablespace separados.
TS_HISTORICO_RESERVAS 64 ;B y USERS 25 MB

Justificación: La decisión de diseño de este paso es por qué se separa el histórico de reservas en un tablespace propio en lugar de dejar las catorce tablas juntas.

La razón es que las tablas no se comportan igual. Nueve de ellas son de catálogo, como municip io, alojamiento, habitación, temporada, tarifa y cliente. Tienen pocas filas, cambian poco y se consultan permanentemente. Las otras cinco forman el histórico, que son reserva, reserva_habitacion, reserva_servicio, pago y resena. Estas concentran cerca del 99 por ciento de las filas de la base, solo crecen con el tiempo y, una vez que la reserva se cierra, ya nadie las modifica.

Separarlas trae cuatro beneficios concretos.
El primero es que permite respaldar cada grupo de forma distinta: el catálogo se puede respaldar completo todas las noches porque es pequeño, mientras que el histórico se respalda de forma incremental, ya que copiar todas las noches veinticinco mil reservas que no cambiaron solo alarga el proceso sin aportar nada.
El segundo es que se puede dar mantenimiento al histórico de forma independiente, por ejemplo dejándolo en modo solo lectura o comprimiendo las temporadas viejas, sin tocar las tablas que la aplicación está usando en ese momento.

El tercero es el aislamiento del espacio, porque si el histórico llega a desbordarse choca contra su propio límite de 2 GB y las tablas de catálogo, que están en otro archivo, siguen funcionando. En un solo tablespace se caería todo al mismo tiempo.
El cuarto es que la recuperación ante una falla es mucho más rápida, ya que restaurar el catálogo toma segundos, mientras que si todo estuviera junto habría que esperar a que se restaure el histórico completo para poder volver a operar.

5. **Comprobar que los permisos quedaron.**

SELECT privilege FROM dba_sys_privs WHERE grantee = 'TURISMOUQ';

```
SQL> SELECT privilege FROM dba_sys_privs WHERE grantee = 'TURISMOUQ';

PRIVILEGE
----------------------------------------
CREATE JOB
CREATE MATERIALIZED VIEW
CREATE VIEW
```

SELECT granted_role FROM dba_role_privs WHERE grantee = 'TURISMOUQ';

```
SQL> SELECT granted_role FROM dba_role_privs WHERE grantee = 'TURISMOUQ';

GRANTED_ROLE
--------------------------------------------------------------------------------
RESOURCE
CONNECT
```

En este paso se consultaron dos vistas del diccionario de datos de Oracle para confirmar que el usuario quedó con lo que necesita. Se usan dos vistas distintas porque Oracle guarda los permisos en dos lugares según su naturaleza.
En dba_sys_privs están los privilegios de sistema, que son permisos sueltos para hacer una cosa puntual, y ahí aparecieron CREATE VIEW, CREATE MATERIALIZED Y CREATE JOB.
Y en dba_role_priv están los toles, que son paquetes de permisos guardados, y ahí aparecieron CONNECT Y RESOURCE, que permite iniciar sesión y el otro permite crear tablas, índices, código PL/SQL dentro del propio sistema.

6. Conectarse como el usuario del proyecto y crear las tablas.

Ahora vamos a salir de administrador, y seguiremos como usuario del sistema, para evitar la mala practica de hacerlo como administrador.

CONNECT turismouq/Daniel.sharond15@localhost:1521/XEPDB1

Con este comando cambiamos de usuario.

Creamos las 14 tablas:
@"C:\Users\HILOS ARMENIA\Desktop\PROYECTO_BASES_DATOS\scripts\01_ddl.sql"

```
============================================================
DDL ejecutado. Tablas creadas en el esquema TURISMOUQ:
============================================================

TABLE_NAME
------------------------------------------------------------
ALOJAMIENTO
CLIENTE
HABITACION
MUNICIPIO
PAGO
RESENA
RESERVA
RESERVA_HABITACION
RESERVA_SERVICIO
SERVICIO
TARIFA

TABLE_NAME
------------------------------------------------------------
TEMPORADA
TIPO_ALOJAMIENTO
USUARIO_SISTEMA

14 filas seleccionadas.
```

Creación de las catorce tablas.

El script 01_ddl.sql empieza con un bloque PL/SQL que recorre las tablas existentes y las borra. Eso hace que el script sea repetible: se puede ejecutar las veces que haga falta y siempre termina en el mismo estado. Así cumplimos con el enunciado, que los scripts corran de arriba abajo sobre una base limpia.

Las tablas se crearon con los cinco tipos de restricción que pide el enunciado. La clave primaria identifica cada fila, la clave foránea garantiza que no exista una habitación apuntando a un alojamiento que no existe, el NOT NULL obliga a que la columna tenga valor, el CHECK valida el contenido, por ejemplo que el estado de una reserva solo pueda ser PENDIENTE, CONFIRMADA, FINALIZADA o CANCELADA y que la fecha de salida sea posterior a la de entrada, y el UNIQUE impide duplicados de negocio como dos

clientes con el mismo documento. En total quedaron 14 claves primarias, 16 foráneas, 11 restricciones de unicidad y 65 entre CHECK y NOT NULL.

Todas las restricciones se nombraron a propósito con prefijos pk, fk, ck, nn y uq. Si no se les pone nombre, Oracle les asigna uno automático como SYS_C0011234, y cuando una inserción falla el mensaje de error no dice cuál regla se violó. Con nombres propios el error apunta directamente a la causa.

Al final el script crea trece índices sobre las llaves foráneas. Esto es necesario porque Oracle crea índice automáticamente sobre la clave primaria y sobre las columnas UNIQUE, pero no sobre las foráneas. Sin ellos, los JOIN de las consultas de análisis harían recorrido completo de tablas de decenas de miles de filas. Los índices de las tablas del histórico se crearon indicando expresamente que van al tablespace TS_HISTORICO_RESERVAS, porque si no se indica se van al tablespace por defecto y se rompería la separación que se justificó en el paso 4.

7. Probar la base de datos.

@"C:\Users\HILOS ARMENIA\Desktop\PROYECTO_BASES_DATOS\scripts\02_carga.sql"

```
============================================================
CARGA TERMINADA - volumen por tabla
============================================================

TABLA                  FILAS
------------------ ----------
MUNICIPIO                  12
TIPO_ALOJAMIENTO            6
ALOJAMIENTO                60
HABITACION                400
TEMPORADA                  28
TARIFA                  11200
CLIENTE                  3000
RESERVA                 25000
RESERVA_HABITACION      37892
SERVICIO                  240
RESERVA_SERVICIO        39952

TABLA                  FILAS
------------------ ----------
PAGO                    22951
RESENA                   9395
USUARIO_SISTEMA           126

14 filas seleccionadas.
```

Se cumplió el volumen mínimo del enunciado: 12 municipios, 60 alojamientos, 400 habitaciones, 3000 clientes, 25000 reservas y 39952 líneas de servicios. Los datos se generan con PL/SQL y DBMS_RANDOM, como pide el proyecto.

Dos decisiones del script. Primera, las 25000 reservas se arman en memoria y se insertan con una sola instrucción FORALL en vez de 25000 INSERT individuales, lo que baja el tiempo de minutos a menos de

un segundo. Segunda, el 45 por ciento de las reservas se fuerza dentro de temporada alta, porque si las fechas fueran uniformes las consultas de ocupación darían una línea plana y no mostrarían la estacionalidad del turismo del Quindío.

Al final el script recolecta estadísticas con DBMS_STATS, sin las cuales el optimizador de Oracle no sabe cuántas filas tiene cada tabla y elige planes de ejecución malos.

8. Ver la decisión de diseño más importante.

```
SELECT COUNT(*) AS cruzan FROM (
  SELECT id_reserva, id_habitacion FROM reserva_habitacion
  GROUP BY id_reserva, id_habitacion HAVING COUNT(*) > 1);
```

```
SQL> SELECT COUNT(*) AS cruzan FROM (
  2    SELECT id_reserva, id_habitacion FROM reserva_habitacion
  3    GROUP BY id_reserva, id_habitacion HAVING COUNT(*) > 1);

    CRUZAN
----------
      3493
```

3493 estadía cruzan de una temporada a otra, o sea el 14%.
Ahora veamos un caso concreto.

```
SET LINESIZE 130
COLUMN temporada FORMAT A20
SELECT * FROM (
  SELECT rh.id_reserva, r.fecha_checkin, r.fecha_checkout, t.nombre AS temporada,
       rh.noches, rh.valor_noche_aplicado
  FROM reserva_habitacion rh
  JOIN reserva r ON r.id_reserva = rh.id_reserva
  JOIN temporada t ON t.id_temporada = rh.id_temporada
  WHERE (rh.id_reserva, rh.id_habitacion) IN (
    SELECT id_reserva, id_habitacion FROM reserva_habitacion
    GROUP BY id_reserva, id_habitacion HAVING COUNT(*) > 1)
  ORDER BY rh.id_reserva, t.fecha_inicio)
WHERE ROWNUM <= 4;
```

```
ID_RESERVA FECHA_CH FECHA_CH TEMPORADA             NOCHES VALOR_NOCHE_APLICADO
---------- -------- -------- -------------------- ---------- --------------------
         4 11/07/24 18/07/24 Mitad de Ano 2024             5               219000
         4 11/07/24 18/07/24 Baja III 2024                 2               133000
         8 31/12/26 06/01/27 Fin de Ano 2026               1               498000
         8 31/12/26 06/01/27 Alta Enero 2027               5               536000
```

La reserva 4 entra el 11/07/24 y sale el 18/07/24, son 7 noches. Cinco caen en Mitad de Año 2024 a 219000 y dos en Baja III 2024 a 133000. La misma habitación, dos tarifas distintas, dos filas.

La reserva 8 es mejor todavía: entra el 31/12/26 y sale el 06/01/27. Una noche en Fin de Año 2026 a 498000 y cinco en Alta Enero 2027 a 536000. Esa fila es la que justifica por qué se definieron temporadas de 2027 aunque las reservas sean de 2024 a 2026: sin ellas esas cinco noches no tendrían tarifa.

El valor de la estadía es simplemente la suma de noches por tarifa, sin recalcular nada.

9. Ejectuar las 8 consultas de análisis.

Consulta 1: Ocupación por municipio y mes, con PIVOT
Calcula que porcentaje de las habitaciones de cada municipio estuvo ocupado cada mes de 2025. La fórmula es noches vendidas dividido entre habitaciones del municipio por días del mes.
PIVOT convierte las filas en columnas. Sin él serían 144 filas, doce municipios por doce meses. Con él son 12 filas y los meses quedan como columnas, que es como se lee un informe de ocupación.
Enero y Diciembre por encima del 50% podemos ver.
Marzo cerca del 40% por semana Santa
Febrero, Mayo y Agosto entre el 10 y el 18%
//Los días del mes se calculan con LAST_DAY y no con un promedio de 30, para que febrero no quede mal medido.

```
SQL> @"C:\Users\HILOS ARMENIA\Desktop\PROYECTO_BASES_DATOS\scripts\03_consultas_analisis.sql"

MUNICIPIO            ENE       FEB       MAR       ABR       MAY       JUN       JUL       AGO       SEP       OCT       NOV       DIC
-------------- ---------- --------- --------- --------- --------- --------- --------- --------- --------- --------- --------- ---------
Armenia              36,1      17,2        35      25,3        18      36,3      31,9      10,2      17,5      14,5      16,2      52,9
Buenavista           46,5      15,2      39,2      31,3      16,9      37,1      29,1      16,6      16,5      18,7      15,9      50,2
Calarca              59,6      15,7      41,7        31      14,7      26,4      36,2        15      18,4      14,4      13,4      50,4
Circasia             55,9      19,4      47,7      35,9      19,7      40,1        36        16      16,3      13,4      13,4      56,5
Cordoba              45,3      14,2      36,6      29,4        18      44,5      38,9      18,8      18,3      16,9      14,3      55,2
Filandia             53,6       9,1      49,9      28,8      17,1      39,1      33,6      17,7      15,4      16,2      16,3      55,1
Genova               56,7      13,2      34,9      28,1      20,6      33,1      38,4      16,7        21      18,8      23,8      56,1
La Tebaida           57,2      12,7      37,1      34,7      15,6      36,9      33,9      13,8      24,3        15      18,9      51,1
Montenegro           63,8      13,1      39,7      33,5      13,9        31      42,9      15,3      15,1      22,4        17      51,1
Pijao                48,9      14,1      36,9      34,6      15,2      29,1      38,3      18,3        12      17,5      22,5      56,3
Quimbaya             53,8      13,1        34      30,8      13,6      40,2      42,4      20,9      14,1      14,4      18,5      53,8
Salento              53,6      14,8      37,8      32,4      11,3      33,1      35,1        21      14,4      14,6        17      49,6

12 filas seleccionadas.
```

Consulta 2:
Ingresos con ROLLUP y GROUPING.
ROLLUP genera los subtotales solos: por tipo de alojamiento, por municipio y el total general, todo en una sola pasada. Sin él se deben escribir 4 consultar y unirlas.
GROUPING devuelve 1 cuando la fila es un subtotal y 0 cuando es un dato real.
Sirve para no confundir un subtotal con un dato cuyo valor fuera nulo.

Eso es lo que llena la columna NIVEL.

MUNICIPIO	TIPO_ALOJAMIENTO	TEMPORADA	RESERVAS	INGRESOS_MILL	NIVEL
Armenia	Apartamento Turistico	ALTA	215	233,4	dato
Armenia	Apartamento Turistico	BAJA	159	99,9	dato
Armenia	Apartamento Turistico	(todas)	355	333,3	subtotal tipo
Armenia	Finca Cafetera	ALTA	107	140,2	dato
Armenia	Finca Cafetera	BAJA	99	72,0	dato
Armenia	Finca Cafetera	(todas)	191	212,1	subtotal tipo
Armenia	Hostal	ALTA	153	100,1	dato
Armenia	Hostal	BAJA	139	69,3	dato
Armenia	Hostal	(todas)	271	169,4	subtotal tipo
Armenia	(todos)	(todas)	817	714,9	subtotal municipio
Filandia	Apartamento Turistico	ALTA	244	170,5	dato
Filandia	Apartamento Turistico	BAJA	160	70,8	dato
Filandia	Apartamento Turistico	(todas)	380	241,4	subtotal tipo
Filandia	Cabana	ALTA	78	62,4	dato
Filandia	Cabana	BAJA	69	34,6	dato
Filandia	Cabana	(todas)	135	97,0	subtotal tipo
Filandia	Finca Cafetera	ALTA	219	322,2	dato
Filandia	Finca Cafetera	BAJA	190	180,3	dato
Filandia	Finca Cafetera	(todas)	374	502,5	subtotal tipo
Filandia	(todos)	(todas)	889	840,8	subtotal municipio
Salento	Apartamento Turistico	ALTA	69	52,5	dato
Salento	Apartamento Turistico	BAJA	63	29,2	dato
Salento	Apartamento Turistico	(todas)	124	81,7	subtotal tipo
Salento	Cabana	ALTA	118	167,3	dato
Salento	Cabana	BAJA	92	83,8	dato
Salento	Cabana	(todas)	189	251,1	subtotal tipo
Salento	Finca Cafetera	ALTA	109	100,4	dato
Salento	Finca Cafetera	BAJA	111	75,6	dato
Salento	Finca Cafetera	(todas)	209	176,0	subtotal tipo
Salento	Hostal	ALTA	104	60,9	dato
Salento	Hostal	BAJA	105	39,0	dato

Consulta 3:

Top 3 alojamientos por municipio, con RANK y PARTITION BY.

PARTITION BY municipio hace que el ranking se reinicie en cada municipio. Por eso ves 1, 2, 3 en Armenia y vuelve a 1 en Buenavista.

Se usa RANK y no ROW_NUMBER porque si dos empatan en ingresos deben quedar en el mismo puesto, no uno arriba del otro por azar.

Y la consulta está envuelta en una subconsulta porque las funciones de ventana se calculan después del GROUP BY, así que no se puede filtrar con WHERE puesto menor o igual a 3 en el mismo nivel.

MUNICIPIO	PUESTO	ALOJAMIENTO	TIPO	RESERVAS	INGRESOS_MILL
Armenia	1	Sendero Verde 25	Apartamento Turistico	652	629,8
Armenia	2	El Mirador 13	Finca Cafetera	522	544,9
Armenia	3	Sendero Verde 1	Apartamento Turistico	449	409,3
Buenavista	1	Buena Vista 48	Finca Cafetera	532	663,5
Buenavista	2	Buena Vista 36	Finca Cafetera	421	533,1
Buenavista	3	Sendero Verde 60	Apartamento Turistico	417	526,3
Calarca	1	Buena Vista 2	Finca Cafetera	561	531,2
Calarca	2	El Mirador 50	Finca Cafetera	491	502,6
Calarca	3	Camino Real 26	Hostal	588	347,4
Circasia	1	La Esperanza 27	Glamping	658	717,7
Circasia	2	Sendero Verde 3	Apartamento Turistico	677	677,1
Circasia	3	Villa Carolina 51	Glamping	485	624,6
Cordoba	1	El Recreo 16	Hotel	556	516,5
Cordoba	2	Alto del Cielo 28	Apartamento Turistico	645	418,0
Cordoba	3	El Cafetal 4	Hostal	685	329,2
Filandia	1	Buena Vista 5	Finca Cafetera	696	1.103,1
Filandia	2	Sendero Verde 29	Apartamento Turistico	649	447,5
Filandia	3	Buena Vista 41	Finca Cafetera	400	336,2
Genova	1	La Esperanza 6	Glamping	719	987,6
Genova	2	Los Guaduales 30	Cabana	657	573,4
Genova	3	La Esperanza 54	Glamping	414	528,2
La Tebaida	1	El Recreo 7	Hotel	672	564,2
La Tebaida	2	La Palma Real 31	Hotel	557	559,5
La Tebaida	3	La Palma Real 19	Hotel	503	539,9
Montenegro	1	Balcones del Quindio 8	Cabana	658	730,2
Montenegro	2	Villa Carolina 20	Glamping	527	512,7
Montenegro	3	Alto del Cielo 32	Apartamento Turistico	474	471,1
Pijao	1	Villa Carolina 21	Glamping	534	678,6
Pijao	2	La Palma Real 9	Hotel	707	619,5
Pijao	3	Sendero Verde 45	Apartamento Turistico	508	572,7
Quimbaya	1	Alto del Cielo 10	Apartamento Turistico	691	848,6
Quimbaya	2	Los Guaduales 34	Cabana	447	567,0

Consulta 4:

Variación de ingresos mes contra mes, con LAG.

LAG trae el valor de la fila anterior sin necesidad de unir la tabla consigo misma. Aquí compara cada mes con el anterior y calcula la diferencia en pesos y en porcentaje.

La primera fila sale vacía porque enero de 2024 no tiene mes anterior. Es correcto, no es un error.

diciembre 2024 sube 445 por ciento contra noviembre, y febrero 2025 cae 83 por ciento contra enero.

MES	INGRESOS_MILL	MES_ANTERIOR	VARIACION	VAR_%
2024-01	1.502,0			
2024-02	306,1	1.502,0	-1.196,0	-79,6
2024-03	1.129,5	306,1	823,5	269,1
2024-04	765,1	1.129,5	-364,4	-32,3
2024-05	327,3	765,1	-437,9	-57,2
2024-06	1.084,6	327,3	757,3	231,4
2024-07	947,0	1.084,6	-137,5	-12,7
2024-08	315,1	947,0	-631,9	-66,7
2024-09	284,1	315,1	-31,0	-9,8
2024-10	315,4	284,1	31,3	11,0
2024-11	316,2	315,4	,8	0,2
2024-12	1.724,6	316,2	1.408,4	445,4
2025-01	1.645,7	1.724,6	-78,9	-4,6
2025-02	267,7	1.645,7	-1.378,0	-83,7
2025-03	1.240,5	267,7	972,8	363,4
2025-04	840,8	1.240,5	-399,7	-32,2
2025-05	341,2	840,8	-499,6	-59,4
2025-06	1.102,4	341,2	761,2	223,1
2025-07	1.104,4	1.102,4	1,9	0,2
2025-08	344,3	1.104,4	-760,1	-68,8
2025-09	344,3	344,3	,0	0,0
2025-10	348,8	344,3	4,6	1,3
2025-11	352,3	348,8	3,5	1,0
2025-12	1.861,2	352,3	1.508,9	428,2
2026-01	1.761,5	1.861,2	-99,8	-5,4
2026-02	340,5	1.761,5	-1.421,0	-80,7
2026-03	1.388,3	340,5	1.047,8	307,8
2026-04	877,5	1.388,3	-510,7	-36,8
2026-05	347,1	877,5	-530,5	-60,4
2026-06	1.267,5	347,1	920,4	265,2
2026-07	1.127,6	1.267,5	-139,9	-11,0
2026-08	396,8	1.127,6	-730,8	-64,8
2026-09	365,2	396,8	-31,6	-8,0

Consulta 5:

Consulta parametrizada con variables de enlace.

Las variables de enlace se escriben con dos puntos delante del nombre. En SQL*Plus se declaran con VARIABLE y se les asigna valor con EXEC, y luego la consulta las recibe dentro del WHERE para filtrar por rango de fechas.

Se usan por dos razones. La primera es que Oracle guarda un solo plan de ejecución y lo reutiliza con cualquier rango, mientras que si el valor se concatenara dentro del texto del SQL cada rango sería una sentencia distinta y obligaría a un análisis nuevo cada vez. La segunda es que el valor viaja como dato y nunca como código, lo que elimina el riesgo de inyección de SQL.

Para cambiar el rango basta con volver a asignar las variables con EXEC y ejecutar la consulta de nuevo, sin modificar una sola línea del SELECT.

MUNICIPIO	TIPO_ALOJAMIENTO	RESERVAS	NOCHES	INGRESOS_MILL
La Tebaida	Hotel	90	390	102,9
Genova	Glamping	66	271	97,8
Buenavista	Finca Cafetera	52	191	77,3
Quimbaya	Apartamento Turistico	51	214	74,9
Circasia	Glamping	59	224	73,6
Filandia	Finca Cafetera	51	194	70,9
Circasia	Apartamento Turistico	61	239	67,8
Pijao	Hotel	60	265	64,5
Montenegro	Glamping	56	236	63,2
Armenia	Apartamento Turistico	53	197	62,2
Calarca	Finca Cafetera	49	221	58,7
Calarca	Hostal	78	313	57,2
Cordoba	Hotel	48	220	54,7
Montenegro	Cabana	39	181	52,9
Filandia	Apartamento Turistico	65	262	52,3

15 filas seleccionadas.

Consulta 6:

Vista materializada de ocupación mensual.

Una vista normal vuelve a ejecutar el JOIN de cinco tablas y veinticinco mil reservas cada vez que alguien la consulta. La vista materializada guarda el resultado en disco como si fuera una tabla, y la consulta pasa a ser un recorrido de unos cientos de filas.

Se definió con BUILD IMMEDIATE, que la calcula al momento de crearla, REFRESH COMPLETE, que recalcula todo al refrescar, y START WITH SYSDATE NEXT SYSDATE mas 1, que programa un refresco automático cada veinticuatro horas.

La política de refresco se justifica en tres puntos. Se eligió COMPLETE y no FAST porque el refresco incremental exige mantener registros de cambios en las cinco tablas del JOIN y además no admite COUNT DISTINCT, que es lo que se necesita para contar reservas; ese costo sería permanente en cada inserción, mientras que recalcular unos cientos de filas en la madrugada toma segundos. Se eligió ON DEMAND y no ON COMMIT porque ON COMMIT recalcularía el agregado dentro de la transacción de cada reserva nueva, encareciendo la operación más frecuente del sistema para beneficiar a un reporte que se consulta una vez al día. Y se eligió frecuencia diaria porque se trata de un indicador mensual de ocupación, donde un desfase de hasta veinticuatro horas no cambia ninguna decisión de negocio.

Un detalle técnico: el refresco automático Oracle lo implementa como una tarea del planificador, por lo que el usuario necesita el privilegio CREATE JOB. Sin él la creación falla con ORA-01031.

MUNICIPIO	MES	RESERVAS	NOCHES_VENDIDAS	INGRESOS_MILL
Armenia	2025-06	91	370	91,7
Armenia	2025-07	84	336	79,0
Armenia	2025-08	28	107	15,3
Buenavista	2025-06	88	367	112,1
Buenavista	2025-07	77	298	91,4
Buenavista	2025-08	42	170	29,2
Calarca	2025-06	67	269	57,1
Calarca	2025-07	92	382	75,6
Calarca	2025-08	43	158	18,0
Circasia	2025-06	105	409	106,5
Circasia	2025-07	88	379	92,1
Circasia	2025-08	37	169	31,3
Cordoba	2025-06	113	454	91,3
Cordoba	2025-07	102	410	78,0
Cordoba	2025-08	48	198	24,1
Filandia	2025-06	101	387	97,8
Filandia	2025-07	88	344	81,9
Filandia	2025-08	39	181	32,4
Genova	2025-06	88	328	89,8
Genova	2025-07	91	393	104,9
Genova	2025-08	44	171	31,6
La Tebaida	2025-06	89	365	95,3
La Tebaida	2025-07	85	347	90,7
La Tebaida	2025-08	37	141	23,6
Montenegro	2025-06	73	307	85,4
Montenegro	2025-07	97	439	108,1
Montenegro	2025-08	40	157	30,3
Pijao	2025-06	76	288	80,3
Pijao	2025-07	91	392	96,9
Pijao	2025-08	52	187	34,6
Quimbaya	2025-06	101	398	118,5

Consulta 7:
UNPIVOT.

Es la operación inversa del PIVOT: pasa columnas a filas. Primero se arma un resumen ancho con una fila por municipio y cuatro columnas, que son noches y clientes en temporada alta y en temporada baja. Luego UNPIVOT lo devuelve a formato largo, con una fila por municipio, métrica y temporada.

En este caso el UNPIVOT es de dos columnas, porque la cláusula FOR recibe métrica y temporada a la vez, de modo que cada columna original se descompone en dos etiquetas.
Sirve porque el formato ancho es cómodo de leer en pantalla pero incómodo para filtrar o graficar. Para pedir todo lo de temporada alta en formato ancho habría que nombrar cada columna, mientras que en formato largo basta un WHERE temporada igual a ALTA. Es la forma normalizada que consumen las herramientas de reportería.

MUNICIPIO	METRICA	TEMPOR	VALOR
Armenia	CLIENTES	ALTA	1.150
Armenia	CLIENTES	BAJA	1.065
Armenia	NOCHES	ALTA	5.433
Armenia	NOCHES	BAJA	4.914
Buenavista	CLIENTES	ALTA	1.144
Buenavista	CLIENTES	BAJA	1.039
Buenavista	NOCHES	ALTA	5.166
Buenavista	NOCHES	BAJA	4.833
Calarca	CLIENTES	ALTA	1.208
Calarca	CLIENTES	BAJA	1.050
Calarca	NOCHES	ALTA	5.737
Calarca	NOCHES	BAJA	4.901
Circasia	CLIENTES	ALTA	1.268
Circasia	CLIENTES	BAJA	1.091
Circasia	NOCHES	ALTA	6.096
Circasia	NOCHES	BAJA	5.141
Cordoba	CLIENTES	ALTA	1.180
Cordoba	CLIENTES	BAJA	1.099
Cordoba	NOCHES	ALTA	5.595
Cordoba	NOCHES	BAJA	5.080
Filandia	CLIENTES	ALTA	1.183
Filandia	CLIENTES	BAJA	1.032
Filandia	NOCHES	ALTA	5.602
Filandia	NOCHES	BAJA	4.833
Genova	CLIENTES	ALTA	1.221
Genova	CLIENTES	BAJA	1.091
Genova	NOCHES	ALTA	5.713
Genova	NOCHES	BAJA	5.142
La Tebaida	CLIENTES	ALTA	1.188
La Tebaida	CLIENTES	BAJA	1.105
La Tebaida	NOCHES	ALTA	5.585
La Tebaida	NOCHES	BAJA	5.210

Consulta 8:
Consulta libre.

La pregunta de negocio es cuáles alojamientos están bien calificados por los huéspedes pero venden pocas noches. Son los candidatos naturales a una campaña de promoción, porque el producto ya es bueno y lo que falta es demanda. Invertir publicidad en un alojamiento mal calificado solo acelera las reseñas negativas.

Para responderla se calcula la posición de cada alojamiento en dos rankings distintos, uno por calificación promedio y otro por noches vendidas, y se resta uno del otro. Una brecha grande y positiva significa buen producto mal vendido.

Se exige un mínimo de veinticinco reseñas mediante la cláusula HAVING, porque un alojamiento con una sola reseña de cinco estrellas encabezaría el ranking de calidad sin que ese promedio signifique nada.

Con eso quedan documentadas las ocho consultas de análisis.

ALOJAMIENTO	MUNICIPIO	TIPO	CALIF_PROM	RESENAS	NOCHES	PUESTO CALIDAD	PUESTO VENTAS	BRECHA
Camino Real 58	Quimbaya	Hostal	4,27	134	536	6	59	+53
El Mirador 40	Cordoba	Finca Cafetera	4,27	194	748	5	50	+45
El Cafetal 38	Calarca	Hostal	4,29	220	863	3	45	+42
El Cafetal 49	Armenia	Hostal	4,31	241	967	1	39	+38
El Recreo 57	Pijao	Hotel	4,21	141	589	22	57	+35
El Mirador 50	Calarca	Finca Cafetera	4,31	253	1.028	2	36	+34
El Cafetal 37	Armenia	Hostal	4,24	205	844	17	48	+31
Camino Real 39	Circasia	Hostal	4,26	214	904	13	41	+28
El Cafetal 47	Salento	Hostal	4,26	262	1.024	12	37	+25
Buena Vista 48	Buenavista	Finca Cafetera	4,27	268	1.070	8	30	+22

10 filas seleccionadas.